MIDNIGHTWEBS · FREE RESOURCE

Website Accessibility Checklist: WCAG 2.2

Accessibility is a legal requirement for many EU services and a practical advantage everywhere else. Use this WCAG 2.2 AA checklist to find the barriers that stop people using your site, fix the highest-impact issues first and keep new ones out.

• Updated November 2026 • 14 min read • PDF

1. Know the rules that apply to you

WCAG 2.2 became a W3C Recommendation in October 2023. Level AA means meeting all Level A and AA criteria: 55 in total, including six new criteria added in 2.2. The European Accessibility Act (EAA) applies from 28 June 2025 to many consumer-facing services, including e-commerce.

- [ ] Treat WCAG 2.2 AA as your working target; it is the practical path to the EAA's accessibility outcomes.
- [ ] Check whether the EAA covers you: e-commerce, banking, e-books, transport and several other consumer services are in scope.
- [ ] Do not assume a non-EU address exempts you if you sell to consumers in the EU.
- [ ] Know the exceptions: microenterprises providing services may be exempt, and disproportionate burden must be documented, not assumed.
- [ ] Check your country's national transposition and enforcement rules; penalties and dates can differ.

Important: the EAA does not name WCAG in the directive, but its requirements align with the same four principles. WCAG 2.2 AA is the clearest way to demonstrate conformance.

2. Perceivable: text alternatives, contrast and structure

People must be able to see or hear your content. Most failures here are cheap to fix and affect every visitor, not only those using assistive technology.

- [x] Give meaningful images useful alt text; use empty alt text for purely decorative images.
- [x] Add captions to video and transcripts to audio; do not rely on autoplay.
- [x] Meet 4.5:1 contrast for normal text and 3:1 for large text, icons and interface boundaries.
- [x] Do not use colour alone to convey meaning; underline links inside body text and keep labels visible.
- [x] Use real headings in order, real lists and tables with header cells; avoid text baked into images.
- [x] Support 200% zoom and reflow at 320 CSS pixels without horizontal scrolling for content.

3. Operable: keyboard, focus, target size and motion

If a task can only be completed with a mouse or a precise drag, some customers cannot complete it at all. WCAG 2.2 added specific requirements here.

- [x] Make every function work with a keyboard in a logical order, and provide a skip link to the main content.
- [x] Keep a clearly visible focus indicator; check that sticky headers, footers and cookie banners do not hide the focused element (2.4.11).

- [ ] Make pointer targets at least 24 by 24 CSS pixels, or provide enough spacing; 44 by 44 is a safer default (2.5.8).
- [ ] Provide a single-pointer alternative for every drag action, such as buttons to reorder a list or adjust a slider (2.5.7).
- [ ] Let users pause, stop or hide moving content, and respect `prefers-reduced-motion`.
- [ ] Remove keyboard traps: menus, modals and embedded widgets must be escapable with standard keys.

4. Understandable: forms, errors, consistency and help

Clear language and predictable behaviour reduce effort for everyone. Forms are where accessibility problems most often become lost sales.

- [ ] Give every input a visible, programmatically associated label; explain required fields in text, not only with an asterisk.
- [ ] Describe errors next to the field, explain what went wrong and how to fix it; never rely on colour alone.
- [ ] Do not ask for the same information twice in one process; autofill it or let users select it (3.3.7).
- [ ] Keep help mechanisms such as contact details or chat in the same relative order across pages (3.2.6).
- [ ] Allow password managers and paste, and offer an alternative to puzzle-only login such as an email link or passkey (3.3.8).
- [ ] Declare the page language, mark language changes, and keep navigation and naming consistent.

5. Robust: code, ARIA and third-party widgets

Assistive technology depends on correct names, roles and states. Native HTML usually does this better than custom script.

- [ ] Use valid, semantic HTML and native controls before reaching for ARIA.
- [ ] If you build a custom widget, expose its name, role and state, and support keyboard interaction.
- [ ] Do not override native semantics or hide focusable content from assistive technology.
- [ ] Test with real screen readers: VoiceOver on macOS/iOS, NVDA on Windows, TalkBack on Android.
- [ ] Audit third-party components: cookie banners, chat widgets, carousels, checkout and booking tools.
- [ ] Announce status messages with a live region without unexpectedly moving focus.

6. Test with tools and with people

Automated scans are useful, but they catch only part of the problem. A site can pass a scanner and still be unusable with a keyboard or screen reader.

- [ ] Run an automated scan to catch missing alt text, contrast failures, empty links and basic structure issues.
- [ ] Complete a keyboard-only pass of the main journeys: home, search, product, cart, checkout, contact and login.
- [ ] Test with a screen reader, at 200% zoom and at 320 CSS pixels wide.
- [ ] Submit forms with errors and check that the message is announced and easy to correct.
- [ ] Include people with disabilities in usability testing where possible; it finds issues no checklist will.

- [ ] Publish an accessibility statement with known limitations, a contact channel and remediation plans.

7. A 30-day remediation plan

Accessibility is easier as a routine than as a one-off project. Fix the barriers that block revenue first, then build checks into design and release.

- [ ] Week 1: confirm legal scope, run an automated scan and a keyboard pass, and list blockers by user impact.
- [ ] Week 2: fix alt text, contrast, labels, focus visibility, cookie banner behaviour and target sizes.
- [ ] Week 3: repair forms, authentication, menus, carousels and third-party embeds; run a screen reader test.
- [ ] Week 4: publish an accessibility statement, train content editors, and add accessibility to design and release checklists.
- [ ] Then schedule a quarterly audit and retest after major template or checkout changes.

> **Automated tools are a smoke test, not a certificate.** Use them to find obvious failures, then verify the important journeys manually and with assistive technology.

Sources and further reading

Guidance in this checklist was checked against the following standards and regulatory sources in November 2026.

- [ ] W3C WAI: What's new in WCAG 2.2
- [ ] W3C WAI: European Union accessibility policy

- [x] European Commission: European Accessibility Act
- [x] EUR-Lex: Accessibility of products and services
- [x] OneTrust: Understanding the EAA and WCAG 2.2
- [x] AccessibleEU: The EAA comes into effect in June 2025